REPUBLIC OF TURKEY
YILDIZ TECHNICAL UNIVERSITY
DEPARTMENT OF COMPUTER ENGINEERING

CLASSIFICATION OF HYPERSPECTRAL IMAGES USING MACHINE LEARNING AND DEEP LEARNING METHODS

22011083 — TAN ERCİYAS
22011084 — KEREM BAYDAR

SENIOR PROJECT

Advisor
Dr. Ahmet ELBİR

June, 2026

ACKNOWLEDGEMENTS

We would like to express our sincere gratitude to our esteemed advisor, Dr. Ahmet ELBİR, for his unwavering support throughout the project.

TAN ERCİYAS
KEREM BAYDAR

TABLE OF CONTENTS

LIST OF ABBREVIATIONS

1D-CNN	1-Dimensional Convolutional Neural Network
3D-CNN	3-Dimensional Convolutional Neural Network
AE	Autoencoder
R&D	Research and Development
CNN	Convolutional Neural Networks
GAN	Generative Adversarial Networks
GPU	Graphics Processing Unit
GUI	Graphical User Interface
HSI	Hyperspectral Images
MRF	Markov Random Fields
PCA	Principal Component Analysis
RF	Random Forest
RNN	Recurrent Neural Networks
SAE	Stacked Autoencoders
SVM	Support Vector Machines
ViT	Vision Transformer

LIST OF FIGURES

LIST OF TABLES

ABSTRACT

CLASSIFICATION OF HYPERSPECTRAL IMAGES USING MACHINE LEARNING AND DEEP LEARNING METHODS

TAN ERCİYAS
KEREM BAYDAR

Department of Computer Engineering
Senior Project

Advisor: Dr. Ahmet ELBİR

BU KISIM İLERİDE DOLDURULACAKTIR

Keywords: BU KISIM İLERİDE DOLDURULACAKTIR

ÖZET

HİPERSPEKTRAL GÖRÜNTÜLERİN MAKİNE ÖĞRENMESİ VE DERİN ÖĞRENME YÖNTEMLERİYLE SINIFLANDIRILMASI

TAN ERCİYAS
KEREM BAYDAR

Bilgisayar Mühendisliği Bölümü
Bitirme Projesi

Danışman: Dr. Ahmet ELBİR

BU KISIM İLERİDE DOLDURULACAKTIR

Anahtar Kelimeler: BU KISIM İLERİDE DOLDURULACAKTIR

1
INTRODUCTION

Hyperspectral images (HSI) are rich data structures collected over a wide range of the electromagnetic spectrum, reflecting the physical and chemical properties of objects on the Earth's surface with high precision. Unlike traditional images, they contain hundreds of narrow spectral bands in each pixel, making them extremely valuable in fields such as agriculture, environmental monitoring, and the defense industry. However, this rich information leads to the problem known as the "curse of dimensionality" and results in significantly high computational costs during the training of machine learning and deep learning models. In addition, the number of labeled samples in hyperspectral datasets is generally very limited, which severely restricts the generalization capability of the developed classification models.

The motivation of this project is to conduct an in-depth investigation into the performance potential of supervised and semi-supervised deep learning architectures, such as CNN, ViT, and Self-Training, against the common challenges of high dimensionality and labeled data scarcity in hyperspectral image classification. The most significant original contribution of the project is the integration of the Curriculum Learning strategy to increase training stability and the development of a user-friendly Graphical User Interface (GUI) operating on a multithreaded architecture. This structure allows the complex models to be testable in real-time by the end-user.

Within the scope of this report; Chapter 2 presents a literature analysis and an evaluation of previous studies related to the project topic. Chapter 3 explains the system design and methodology, extending from data preprocessing to model architectures. Chapter 4 analyzes the obtained experimental results comparatively using various performance metrics, and Chapter 5 discusses the project results and provides suggestions for future work.

1.1 Purpose

The primary objective of this project is to quantitatively demonstrate the performance differences between supervised and semi-supervised learning approaches in the classification of hyperspectral images with high-dimensional and complex spectral data. In line with this main goal, the sub-objectives are as follows:

- To analyze model performances on three different fundamental datasets that are considered standards in the literature: Indian Pines, Pavia University, and Salinas.
- To examine the effects of data preprocessing steps, such as Gaussian filtering for noise reduction and Principal Component Analysis (PCA) for dimensionality reduction, on classification success.
- To integrate the Curriculum Learning strategy into the models to maximize stability during the training processes.
- To conduct an objective comparison of the models using Accuracy, Precision, Recall, F1-Score, and Kappa metrics.
- To develop a multithreaded Graphical User Interface (GUI) based on React and Python to prevent the developed models from being confined to desktop or laboratory environments and to demonstrate real-time prediction capabilities.

1.2 Preliminary Investigation

Preliminary investigations conducted during the study of the project area show that traditional machine learning methods are rapidly giving way to deep learning-based approaches in the hyperspectral image classification literature. Research into the gaps of previous studies reveals that existing systems can generally only produce successful results with large amounts of labeled data. However, in real-world scenarios, the labeling of pixels by experts is extremely costly and time-consuming.

The path to be followed in this project targets this gap in the literature by demonstrating how semi-supervised algorithms can overcome this bottleneck by utilizing a small amount of labeled data and a large amount of unlabeled data together. Furthermore, another issue identified in current systems is that models developed by researchers are usually only testable at the code level. This project directly addresses the need for converting theoretical successes into a practical and user-oriented application by introducing an interface that runs multithreaded model instances, enabling users to load external data and receive instantaneous predictions.

2
LITERATURE ANALYSIS

The problem of hyperspectral image (HSI) classification has been one of the most researched topics in the remote sensing literature for many years, aiming to increase object recognition sensitivity by utilizing rich spectral data content. In this process, which extends from traditional machine learning algorithms to modern deep learning-based architectures, many different methods have been proposed, and solutions have been sought for the "curse of dimensionality" and limited labeled data problems. In this section, the approaches in the literature are examined in a comparative manner, grouped according to their algorithmic foundations.

2.1 Traditional Machine Learning Approaches

Early studies in the classification of hyperspectral images predominantly relied on statistical and traditional machine learning methods such as Support Vector Machines (SVM) and Random Forest (RF). Du et al. achieved high accuracy rates (97.71% for Indian Pines) using discriminative sparse representations and multinomial logistic regression [1]. Similarly, Xia et al. attempted to optimize the performance of RF models by applying extended profiles and ensemble strategies [2]. Furthermore, cascaded random forest (Cascaded RF) architectures, where multiple classifiers are trained simultaneously, were proposed to prevent overfitting [3]. Among SVM-based approaches, studies combining probabilistic SVMs with guided filters [4] and integrating spectral-spatial information with Markov Random Fields (MRF) [5] stand out. Although these methods are relatively resistant to noise and offer interpretability, they have fallen behind deep learning algorithms in terms of performance due to their difficulties in dealing with high-dimensional data and the necessity of extracting complex spatial features manually (hand-crafted).

2.2 Convolutional Neural Networks (CNN) Based Approaches

The capacity of Convolutional Neural Networks (CNNs) to process both spatial and spectral information simultaneously has made them a standard in the literature. Chen et al. aimed to solve the high dimensionality problem using 3D Convolutional Neural Networks (3D-CNN) and achieved 99.66% success on the Pavia University dataset with a model supported by L2 norm and dropout techniques [6]. Li et al. proposed a 1D-CNN structure that generates deep pixel-pair features to overcome the limited training data problem [7]. To reduce the high number of parameters and computational costs of 3D-CNN models, various architectures have been developed, such as HybridSN [8], which combines 3D and 2D convolution layers; fast and compact 3D-CNN architectures [9]; and next-generation networks that prevent data loss with border mirroring techniques [10]. Although these methods successfully extract spectral-spatial features, the requirement for large amounts of labeled data and high GPU memory costs as model depth increases remain their primary disadvantages.

2.3 Autoencoders, Generative Networks, and RNNs

The requirements for dimensionality reduction and dealing with limited labeled samples have encouraged the use of Autoencoders (AE), Generative Adversarial Networks (GANs), and Recurrent Neural Networks (RNNs) in the literature. Li et al. proposed a method combining Stacked Autoencoders (SAE) with CNNs to adaptively weight local spatial context [11]. Similarly, Zhao et al. integrated SAEs for dimensionality reduction with 3D deep residual networks [12]. RNNs, which are successful in time series, were adapted by Shi et al. to fuse multi-scale spectral-spatial features [13]. On the other hand, GANs have been utilized to artificially augment limited training data with synthetic samples in an attempt to increase the generalization capability of models [14]. Although these architectures improve accuracy in certain areas, their practical use has been limited by scalability issues arising from the sequential nature of processing and the highly unstable training processes of generative networks.

2.4 Transformer Architectures and Semi-Supervised Methods

The Transformer architecture, one of the most innovative developments in the field of deep learning in recent years, has begun to be used in HSI classification due to its ability to model long-range dependencies. Hybrid models combining the local feature extraction capabilities of CNNs with the global attention mechanisms of Transformers [15, 16], systems based on spectral-spatial feature tokenization [17],

and dual attention networks [18] have come to the fore. Although these methods achieve over 99% success on various datasets, they require extreme computational power compared to traditional machine learning and CNN methods.

Another successful alternative against labeled data scarcity is the semi-supervised 3D deep neural networks examined by Sellami et al. [19]. This study demonstrated the potential of incorporating unlabeled data into the process with adaptive band selection. However, the risk of bias created by these systems while drawing decision boundaries and the difficulty of selecting optimum semi-supervised threshold values remain ongoing problems.

2.5 Current Standing in Literature and Original Value of the Study

Considering the literature reviewed, it is clearly observed that Transformer and deep CNN-based models achieving high accuracy are prone to overfitting under limited labeled data conditions and demand high hardware resources. Although semi-supervised learning and generative models attempt to offer solutions to these hardware and data bottlenecks, the lack of integrated strategies that guarantee model training stability is noteworthy.

In this direction, our project aims to gather scattered algorithmic advantages into a single semi-supervised framework by combining CNN and Transformer-based architectures with Self-Training strategies and the "Curriculum Learning" strategy, which will address the unbalanced learning problems of models in the literature. Unlike complex models that are generally left as mere Python scripts or laboratory codes, our project will provide a user-friendly Graphical User Interface (GUI) based on React and Python capable of simultaneous inference (multithreaded inference). In this way, the high accuracy potential identified in the literature will be transformed from a purely academic result into a testable and practical structure.

3
SYSTEM ANALYSIS AND FEASIBILITY

The purpose of system analysis is to identify and define the main elements and functions to find the most appropriate solution for the project. In this section, the objectives of the project are detailed, and information sources and requirements are determined. By the end of this chapter, all requirements related to the project are defined, and the most suitable solution for proceeding to the design phase is established.

The research and information gathering methods used for eliciting requirements and their results are provided in this section. In light of the determined requirements, the modules, users, and roles in the system are defined. System analysis also reveals the structure to test whether the needs are met upon project completion. For this reason, the performance metrics (speed, recognition success, etc.) to be used in the project are also defined in this section.

3.1 System Analysis

Due to the R&D nature of this project, the fundamental requirements were determined during the system analysis phase by examining the performance bottlenecks of existing deep learning-based hyperspectral classification models. The main function of the system is to process high-dimensional spectral data—either uploaded externally or selected from standard literature datasets such as Indian Pines [20], Pavia University [21], and Salinas [22]—to predict the class of each pixel with high accuracy.

The fundamental requirements that the system is expected to fulfill are as follows:

- De-noising of hyperspectral images using Gaussian filtering and dimensionality reduction through the Principal Component Analysis (PCA) method [23].
- Enabling the training of supervised (CNN [6], ViT [24]) and semi-supervised (Self-Training [25]) models using a Curriculum Learning [26] strategy.

- Providing a graphical user interface (GUI) operating on a multithreaded architecture to allow simultaneous testing and real-time prediction of developed models [27].

To test whether the requirements are met at the end of the project, the following performance metrics will be utilized: Accuracy, Precision, Recall, F1-Score, and Kappa coefficient.

3.2 Feasibility

The feasibility study evaluates the viability of the project by analyzing how resources were selected among alternatives and the project's timeline.

3.2.1 Technical Feasibility

Technical feasibility is a brand-independent study that highlights technical features, including software and hardware feasibility. Python [28] has been selected as the application development language due to its robust ecosystem in the field of machine learning. For deep learning processes, PyTorch [29] or TensorFlow [30] libraries will be used, while Scikit-learn [23] and OpenCV [31] will be utilized for data processing. For the front-end development of the GUI, React [27] has been preferred for its performance and concurrent processing capabilities. The development environment will be VS Code [32], and version control will be managed via GitHub [33].

3.2.1.1 Hardware Feasibility

The high parallel processing capacity required for training deep learning models will be met using the GPU accelerators provided by the Google Colab [34] environment. For front-end development and interface integration, the hardware owned by the project team (macOS-based computers) possesses sufficient capacity.

3.2.2 Legal Feasibility

The tools used in the project, such as Python, React, PyTorch, and OpenCV, have open-source licenses. The Indian Pines, Pavia University, and Salinas datasets are open for academic use. Therefore, it has been determined that the project does not pose any legal barriers or patent infringements.

3.2.3 Economic Feasibility

All libraries and development platforms selected for the project consist of free and open-source tools. Thanks to the use of Google Colab and existing personal hardware, no additional investment costs are incurred, and the project is considered economically viable.

3.2.4 Workforce and Time Feasibility

The project is conducted by Tan Erciyas and Kerem Baydar. The technical expertise of the team covers the necessary domains for project completion. The following Gantt chart illustrates the project timeline from February 2026 to June 2026.

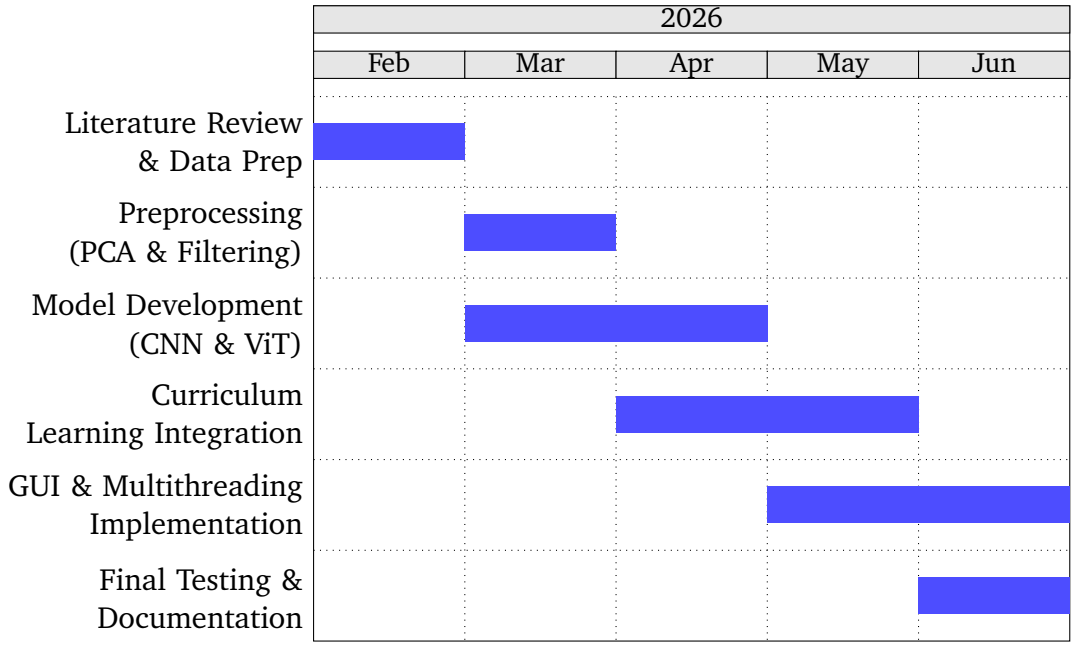

Figure 3.1 Project Gantt Chart (Feb 2026 - June 2026)

4
SYSTEM DESIGN

The system design phase constitutes the core of this project, defining the architectural framework and procedural flow required to process high-dimensional hyperspectral data. This section provides a comprehensive overview of the materials used, the preprocessing pipeline, the deep learning architectures, and the novel learning strategies implemented to achieve high classification accuracy under limited supervision.

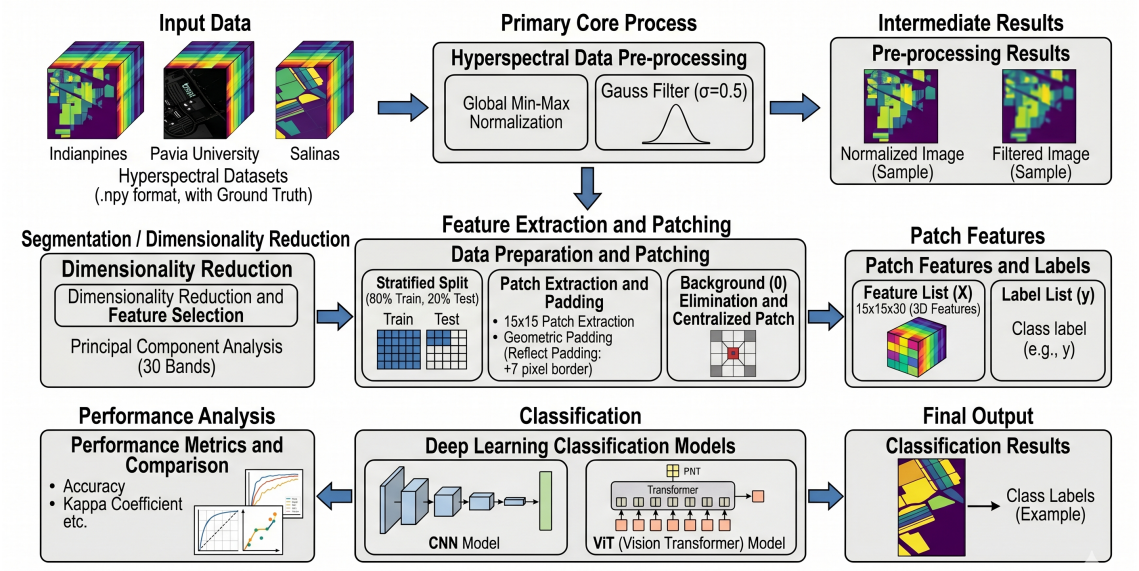

Figure 4.1 Overall Workflow of the Proposed HSI Classification System

4.1 Material and Dataset

The efficacy of hyperspectral image (HSI) classification models is heavily dependent on the quality and characteristics of the datasets used for training and validation. In this project, three benchmark datasets recognized in remote sensing literature are utilized: Indian Pines, Pavia University, and Salinas Scene.

4.1.1 Dataset Specifications

4.1.1.1 Indian Pines

Collected by the AVIRIS sensor over North-Western Indiana, this dataset consists of 145×145 pixels and 224 spectral reflectance bands. It contains 16 classes of vegetation and land cover. Due to water absorption, the number of bands is typically reduced to 200 [20].

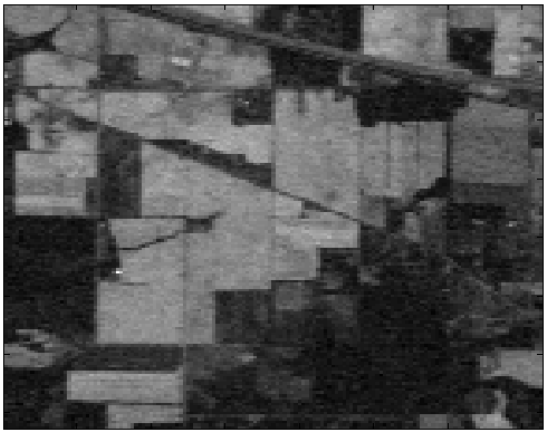

Figure 4.2 Indian Pines – False-color band composite

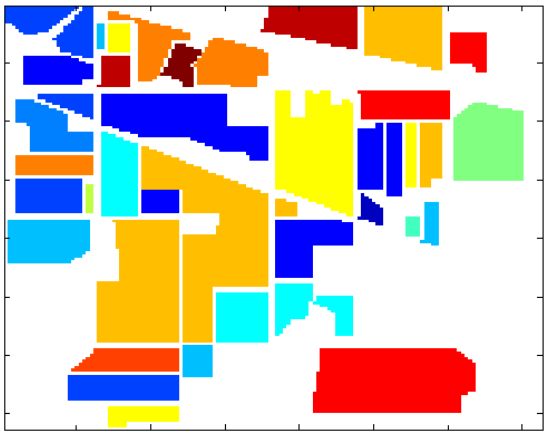

Figure 4.3 Indian Pines – Ground truth map

Table 4.1 Groundtruth classes for the Indian Pines scene and their respective samples number

#	**Class**	**Samples**
1	Alfalfa	46
2	Corn-notill	1428
3	Corn-mintill	830
4	Corn	237
5	Grass-pasture	483
6	Grass-trees	730
7	Grass-pasture-mowed	28
8	Hay-windrowed	478
9	Oats	20
10	Soybean-notill	972
11	Soybean-mintill	2455
12	Soybean-clean	593
13	Wheat	205
14	Woods	1265
15	Buildings-Grass-Trees-Drives	386
16	Stone-Steel-Towers	93
	Total	**10249**

4.1.1.2 Pavia University

Acquired by the ROSIS sensor over an urban area in Italy, it consists of 610 × 340 pixels with 103 spectral bands. It features 9 urban land-cover classes such as asphalt, meadows, and bricks [21].

Figure 4.4 Pavia University – False-color band composite

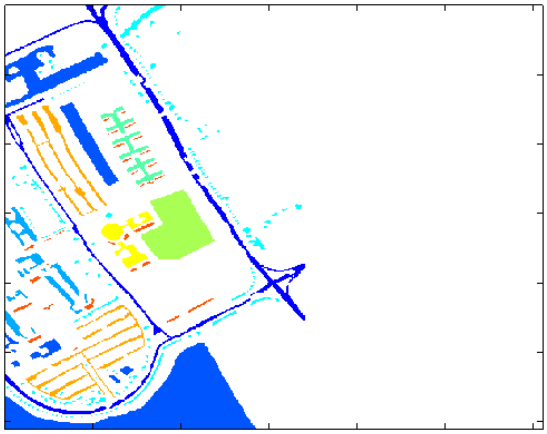

Figure 4.5 Pavia University – Ground truth map

Table 4.2 Groundtruth classes for the Pavia University scene and their respective samples number

#	**Class**	**Samples**
1	Asphalt	6631
2	Meadows	18649
3	Gravel	2099
4	Trees	3064
5	Painted metal sheets	1345
6	Bare Soil	5029
7	Bitumen	1330
8	Self-Blocking Bricks	3682
9	Shadows	947
	Total	**42776**

4.1.1.3 Salinas Scene

Also collected by AVIRIS over Salinas Valley, California, this dataset features high spatial resolution with 512×217 pixels and 224 spectral bands (reduced to 204 after removing water absorption bands) [22].

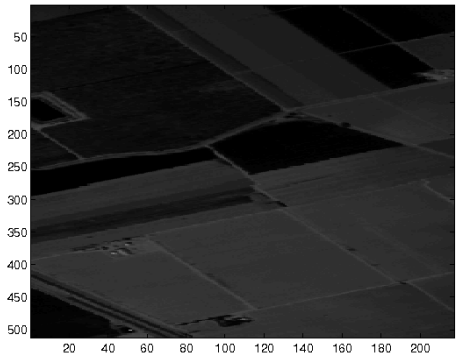

Figure 4.6 Salinas Scene – False-color band composite

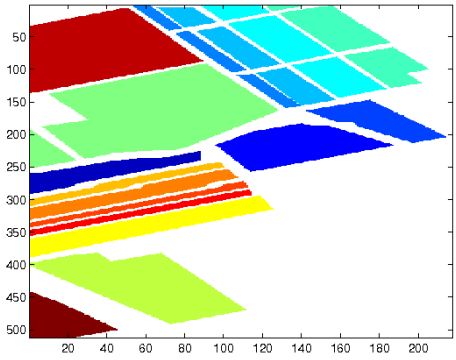

Figure 4.7 Salinas Scene – Ground truth map

Table 4.3 Groundtruth classes for the Salinas scene and their respective samples number

#	**Class**	**Samples**
1	Brocoli_green_weeds_1	2009
2	Brocoli_green_weeds_2	3726
3	Fallow	1976
4	Fallow_rough_plow	1394
5	Fallow_smooth	2678
6	Stubble	3959
7	Celery	3579
8	Grapes_untrained	11271
9	Soil_vinyard_develop	6203
10	Corn_senesced_green_weeds	3278
11	Lettuce_romaine_4wk	1068
12	Lettuce_romaine_5wk	1927
13	Lettuce_romaine_6wk	916
14	Lettuce_romaine_7wk	1070
15	Vinyard_untrained	7268
16	Vinyard_vertical_trellis	1807
	Total	**54129**

4.1.2 Data Structure and Storage

The data is structured as 3D hypercubes ($H \times W \times B$), where H and W represent spatial dimensions and B represents the spectral bands. To optimize memory usage during training, the datasets are stored in NumPy array formats ('.npy') and processed in patches to preserve local spatial relationships.

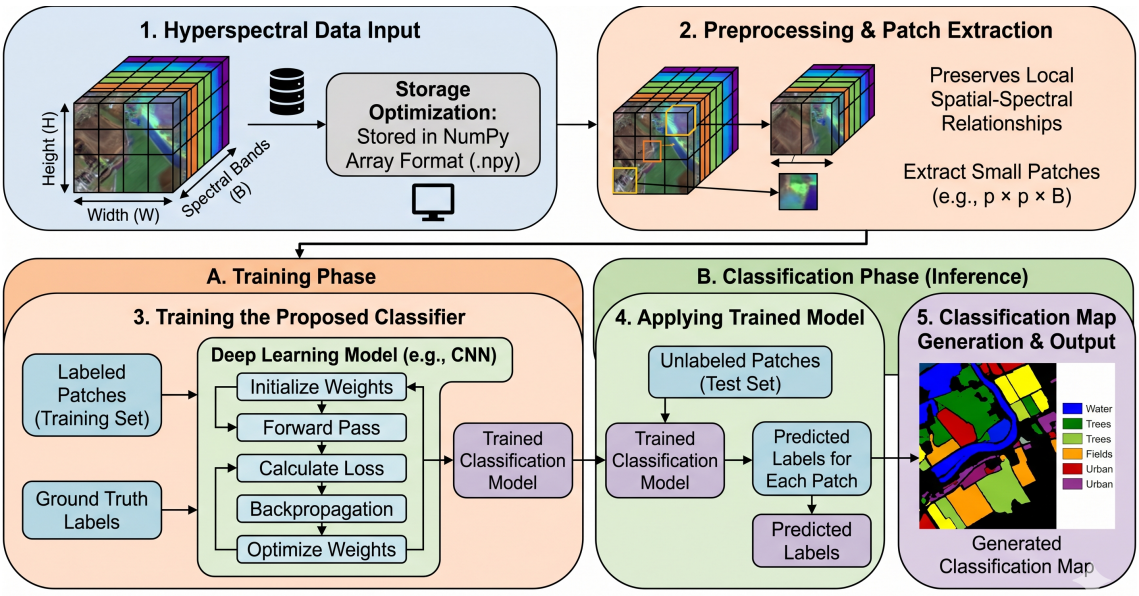

Figure 4.8 General Block Diagram of the Proposed HSI Classification System

4.2 Methodology

The proposed system follows a modular pipeline: preprocessing, feature extraction, and a semi-supervised learning phase supported by curriculum learning.

4.2.1 Preprocessing and Dimensionality Reduction

To address the "curse of dimensionality" inherent in hyperspectral data, the following sequential steps are applied to all three datasets.

1. **Global Min-Max Normalization:** The raw hypercube is reshaped to a 2D matrix and a single global minimum and maximum is computed across all pixels and all bands. Each value is then rescaled to $[0, 1]$ as follows:

$$\hat{x} = \frac{x - x_{\min}}{x_{\max} - x_{\min}} \tag{4.1}$$

   Band-wise normalization was deliberately avoided to preserve the natural variance ratio between bands.

2. **Spatial Gaussian Filtering:** A Gaussian filter is applied with $\sigma = 0.5$ along only the spatial axes ($H \times W$) of the normalized hypercube using `scipy.ndimage.gaussian_filter` with `sigma=(0.5, 0.5, 0)` and `mode='reflect'`. Setting the spectral axis sigma to zero ensures that inter-band relationships remain intact while sensor-induced spatial noise is attenuated.

3. **Fixed-Component PCA:** PCA with a fixed $n\_components = 30$ is applied to the filtered data after reshaping it to $(H \times W, B)$. The variance-based threshold (99.5%) was evaluated first but yielded only 4–5 components for Pavia University and Salinas, which is insufficient for the CNN and ViT input depths. The resulting reduced cube is reshaped back to $(H, W, 30)$ [23].

4.2.2 Patch Extraction and Dataset Splitting

For each labeled pixel (r, c) where `gt[r,c] > 0`, a 15×15 spatial patch is extracted from the PCA-reduced cube, producing a sample of shape $15 \times 15 \times 30$. Background pixels (label $= 0$) are excluded entirely. To allow boundary pixels to serve as valid patch centers, the cube is first padded by $\lfloor 15/2 \rfloor = 7$ pixels on each spatial side using reflect padding.

The resulting patches are split into training and test sets with `test_size=0.2`, `stratify=y`, and `random_state=42`, ensuring minority classes are proportionally represented in both splits. The resulting sample counts are given in Table 4.4.

Table 4.4 Patch counts after stratified 80/20 train-test split ($15 \times 15 \times 30$)

Dataset	Total	Train (80%)	Test (20%)
Indian Pines	10249	8199	2050
Pavia University	42776	34220	8556
Salinas Scene	54129	43303	10826

4.2.3 Model Architectures

The system implements two state-of-the-art architectures to evaluate performance differences:

- **3D-CNN:** This model employs three-dimensional kernels to extract spectral-spatial features simultaneously. The architecture includes volumetric

convolution layers followed by batch normalization and ReLU activation to prevent vanishing gradients [6].

- **Vision Transformer (ViT):** To capture long-range dependencies across spectral bands, the HSI patches are flattened into sequences of embeddings. A self-attention mechanism is applied to weigh the importance of different spectral-spatial tokens [24].

4.2.4 Semi-Supervised Learning and Curriculum Learning

The core innovation of the project lies in handling labeled data scarcity:

- **Self-Training:** The model initially trains on a small set of labeled pixels. It then iteratively predicts labels for the unlabeled pool, incorporating high-confidence samples back into the training set [25].
- **Curriculum Learning (CL):** To ensure training stability, a CL strategy is implemented where the model is first presented with "easy" samples (pixels with high spectral purity and clear spatial context) before transitioning to "difficult" samples near decision boundaries [26].

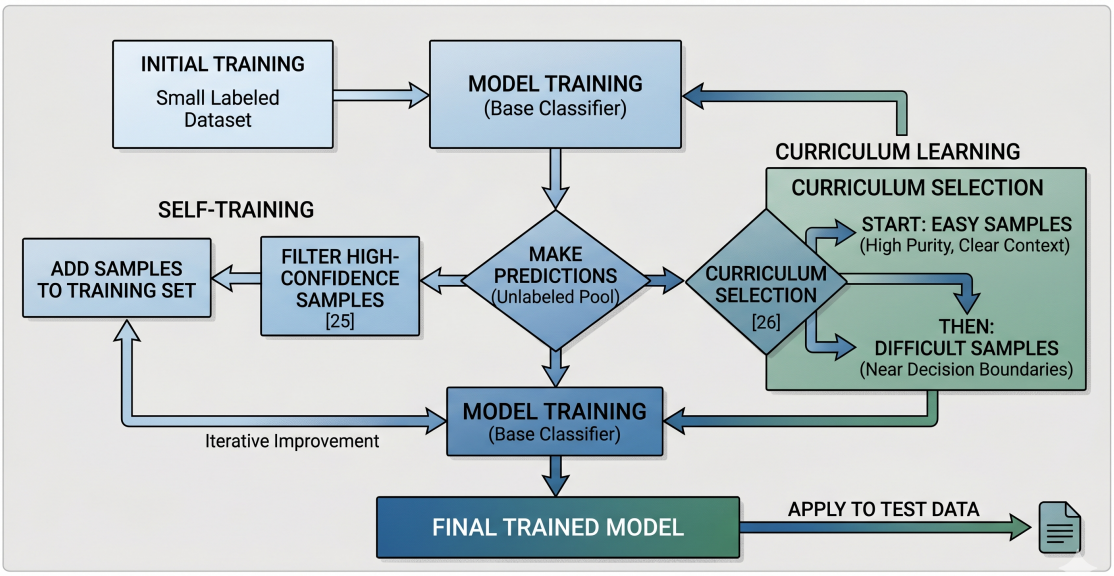

Figure 4.9 Flowchart of the Semi-Supervised Learning Strategy

4.2.5 Evaluation Metrics

To assess model performance, five standard metrics are employed. Let TP_c, FP_c, and FN_c denote the true positives, false positives, and false negatives for class c, and let N be the total number of test samples.

- **Overall Accuracy (OA):** The ratio of correctly classified pixels to the total number of test pixels:
$$\text{OA} = \frac{\sum_c TP_c}{N} \tag{4.2}$$
- **Precision:** The fraction of pixels predicted as class c that truly belong to class c, averaged across all classes (macro):
$$\text{Precision} = \frac{1}{C}\sum_{c=1}^{C}\frac{TP_c}{TP_c + FP_c} \tag{4.3}$$
- **Recall:** The fraction of true class-c pixels that are correctly identified:
$$\text{Recall} = \frac{1}{C}\sum_{c=1}^{C}\frac{TP_c}{TP_c + FN_c} \tag{4.4}$$
- **F1-Score:** The harmonic mean of Precision and Recall:
$$\text{F1} = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \tag{4.5}$$
- **Cohen's Kappa (κ):** A metric that corrects Overall Accuracy for agreement expected by chance. Given the observed accuracy p_o and the chance agreement p_e:
$$\kappa = \frac{p_o - p_e}{1 - p_e} \tag{4.6}$$
A value of $\kappa = 1$ indicates perfect classification, while $\kappa = 0$ indicates performance equivalent to random chance.

4.2.6 GUI and Multithreading Integration

To provide a practical application, a React-based front-end and a Python/FastAPI back-end are developed. To ensure a responsive user experience during heavy model inference, a multithreading approach is utilized. The inference engine runs in a separate thread from the I/O operations, allowing the user to interact with the GUI while the deep learning models process large hyperspectral hypercubes in the background.

References

[1] P. Du, Z. Xue, J. Li, and A. Plaza, "Learning discriminative sparse representations for hyperspectral image classification," *IEEE Journal of Selected Topics in Signal Processing*, vol. 9, no. 6, pp. 1089–1104, 2015.

[2] J. Xia, P. Ghamisi, N. Yokoya, and A. Iwasaki, "Random forest ensembles and extended multiextinction profiles for hyperspectral image classification," *IEEE Transactions on Geoscience and Remote Sensing*, vol. 56, no. 1, pp. 202–216, 2017.

[3] Y. Zhang, G. Cao, X. Li, and B. Wang, "Cascaded random forest for hyperspectral image classification," *IEEE journal of selected topics in applied earth observations and remote sensing*, vol. 11, no. 4, pp. 1082–1094, 2018.

[4] C. Zhang, M. Han, and M. Xu, "Multi-feature classification of hyperspectral image via probabilistic svm and guided filter," in *2018 International Joint Conference on Neural Networks (IJCNN)*, IEEE, 2018, pp. 1–7.

[5] X. Cao, X. Wang, D. Wang, J. Zhao, and L. Jiao, "Spectral–spatial hyperspectral image classification using cascaded markov random fields," *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing*, vol. 12, no. 12, pp. 4861–4872, 2019.

[6] Y. Chen, H. Jiang, C. Li, X. Jia, and P. Ghamisi, "Deep feature extraction and classification of hyperspectral images based on convolutional neural networks," *IEEE transactions on geoscience and remote sensing*, vol. 54, no. 10, pp. 6232–6251, 2016.

[7] W. Li, G. Wu, F. Zhang, and Q. Du, "Hyperspectral image classification using deep pixel-pair features," *IEEE Transactions on Geoscience and Remote Sensing*, vol. 55, no. 2, pp. 844–853, 2016.

[8] S. K. Roy, G. Krishna, S. R. Dubey, and B. B. Chaudhuri, "Hybridsn: Exploring 3-d–2-d cnn feature hierarchy for hyperspectral image classification," *IEEE geoscience and remote sensing letters*, vol. 17, no. 2, pp. 277–281, 2019.

[9] M. Ahmad, A. M. Khan, M. Mazzara, S. Distefano, M. Ali, and M. S. Sarfraz, "A fast and compact 3-d cnn for hyperspectral image classification," *IEEE Geoscience and Remote Sensing Letters*, vol. 19, pp. 1–5, 2020.

[10] M. E. Paoletti, J. M. Haut, J. Plaza, and A. Plaza, "A new deep convolutional neural network for fast hyperspectral image classification," *ISPRS journal of photogrammetry and remote sensing*, vol. 145, pp. 120–147, 2018.

[11] S. Li, X. Zhu, Y. Liu, and J. Bao, "Adaptive spatial-spectral feature learning for hyperspectral image classification," *IEEE access*, vol. 7, pp. 61 534–61 547, 2019.

[12] J. Zhao, L. Hu, Y. Dong, L. Huang, S. Weng, and D. Zhang, "A combination method of stacked autoencoder and 3d deep residual network for hyperspectral image classification," *International Journal of Applied Earth Observation and Geoinformation*, vol. 102, p. 102 459, 2021.

[13] C. Shi and C.-M. Pun, "Multi-scale hierarchical recurrent neural networks for hyperspectral image classification," *Neurocomputing*, vol. 294, pp. 82–93, 2018.

[14] L. Zhu, Y. Chen, P. Ghamisi, and J. A. Benediktsson, "Generative adversarial networks for hyperspectral image classification," *IEEE Transactions on Geoscience and Remote Sensing*, vol. 56, no. 9, pp. 5046–5063, 2018.

[15] X. He, Y. Chen, and Z. Lin, "Spatial-spectral transformer for hyperspectral image classification," *Remote Sensing*, vol. 13, no. 3, p. 498, 2021.

[16] P. Zhang, H. Yu, P. Li, and R. Wang, "Transhsi: A hybrid cnn-transformer method for disjoint sample-based hyperspectral image classification," *Remote Sensing*, vol. 15, no. 22, p. 5331, 2023.

[17] L. Sun, G. Zhao, Y. Zheng, and Z. Wu, "Spectral–spatial feature tokenization transformer for hyperspectral image classification," *IEEE Transactions on Geoscience and Remote Sensing*, vol. 60, pp. 1–14, 2022.

[18] Z. Shu, Y. Wang, and Z. Yu, "Dual attention transformer network for hyperspectral image classification," *Engineering Applications of Artificial Intelligence*, vol. 127, p. 107 351, 2024.

[19] A. Sellami, M. Farah, I. R. Farah, and B. Solaiman, "Hyperspectral imagery classification based on semi-supervised 3-d deep neural network and adaptive band selection," *Expert Systems with Applications*, vol. 129, pp. 246–259, 2019.

[20] M. F. Baumgardner, L. L. Biehl, and D. A. Landgrebe, "220 band aviris hyperspectral image data set: June 12, 1992 indian pine test site 3," *Purdue University Research Repository*, 2015.

[21] P. Dell'Acqua, P. Gamba, et al., "A comparison of svm and rbf neural networks for the classification of hyperspectral images," *IEEE Transactions on Geoscience and Remote Sensing*, 2005.

[22] D. A. Landgrebe et al., *Salinas hyperspectral image dataset*, 1998.

[23] F. Pedregosa et al., "Scikit-learn: Machine learning in python," *the Journal of machine Learning research*, vol. 12, pp. 2825–2830, 2011.

[24] A. Dosovitskiy et al., "An image is worth 16x16 words: Transformers for image recognition at scale," in *International Conference on Learning Representations*, 2021.

[25] I. Triguero, S. García, and F. Herrera, "Self-labeled techniques for semi-supervised learning: Taxonomy, software and empirical study," *Knowledge and Information Systems*, vol. 42, no. 2, pp. 245–284, 2015.

[26] Y. Bengio, J. Louradour, R. Collobert, and J. Weston, "Curriculum learning," in *Proceedings of the 26th annual international conference on machine learning*, 2009, pp. 41–48.

[27] A. Fedosejev, *React. js essentials*. Packt Publishing Ltd, 2015.

[28] G. Van Rossum and F. L. Drake Jr, *Python tutorial*. Centrum voor Wiskunde en Informatica Amsterdam, The Netherlands, 1995.

[29] A. Paszke et al., "Pytorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems 32*, 2019, pp. 8024–8035.

[30] M. Abadi et al., "Tensorflow: A system for large-scale machine learning," in *12th USENIX symposium on operating systems design and implementation (OSDI 16)*, 2016, pp. 265–283.

[31] Itseez, *Open source computer vision library*, `https://github.com/itseez/opencv`, 2015.

[32] Microsoft, *Visual studio code*, `https://code.visualstudio.com/`, 2015.

[33] L. Dabbish, C. Stuart, J. Tsay, and J. Herbsleb, "Social coding in github: Transparency and collaboration in an open software repository," in *Proceedings of the ACM 2012 conference on computer supported cooperative work*, 2012, pp. 1277–1286.

[34] E. Bisong, "Google colaboratory," in *Building Machine Learning and Deep Learning Models on Google Cloud Platform: A Comprehensive Guide for Beginners*, Springer, 2019, pp. 59–64.

Curriculum Vitae

FIRST MEMBER

Name-Surname: TAN ERCİYAS
Birthdate and Place of Birth: 15.12.2003, Trabzon
E-mail: tan.erciyas@std.yildiz.edu.tr
Phone: 0505 353 60 58
Practical Training: Genarion Software Department
Baykar Web Software Department
Tüpraş Software Department

SECOND MEMBER

Name-Surname: KEREM BAYDAR
Birthdate and Place of Birth: 06.11.2004, Kocaeli
E-mail: kerem.baydar@std.yildiz.tr
Phone: 0544 735 41 00
Practical Training: Baykar OOP Software Department
IBTECH Dw & Data Insights Department
Genarion Software Department
Uzer Makina IT Department

Project System Informations

System and Software: Windows İşletim Sistemi, Python
Required RAM: 2GB
Required Disk: 256MB